Review OF OOP And UML

Presentation By Parsa Attaran
Instructor: Dr.Mojtaba Vahidi Asl
1405 Ap Spring

TABLE OF CONTENTS

01

Why OOP?

INTRODUCTION

Object-oriented programming (**OOP**) is a programming paradigm based on the concept of “objects”, which can contain data, in the form of fields or attributes, and code, in the form of methods or functions. A feature of objects is an object’s methods that can access and often modify the data fields of the object with which they are associated (objects have a notion of “this” or “self”). In OOP, computer programs are designed by making them out of objects that interact with one another. OOP languages are diverse, but the most popular ones are class-based, meaning that objects are instances of classes, which also determine their types.

Historical Timeline of the Formation

1960s – The Software Complexity Crisis

In the **1960s and 1970s**, most software was written using **Procedural Programming**. In this approach, programs were organized as a collection of **functions and procedures** that operated on data.
As software systems grew larger, several major problems appeared:

- Difficulty maintaining and updating programs
- High dependency between different parts of the code
- Increased number of errors in large projects
- Difficulty modeling complex real-world systems

These challenges led researchers to search for a new way to organize and structure software.

So what were the challenges?

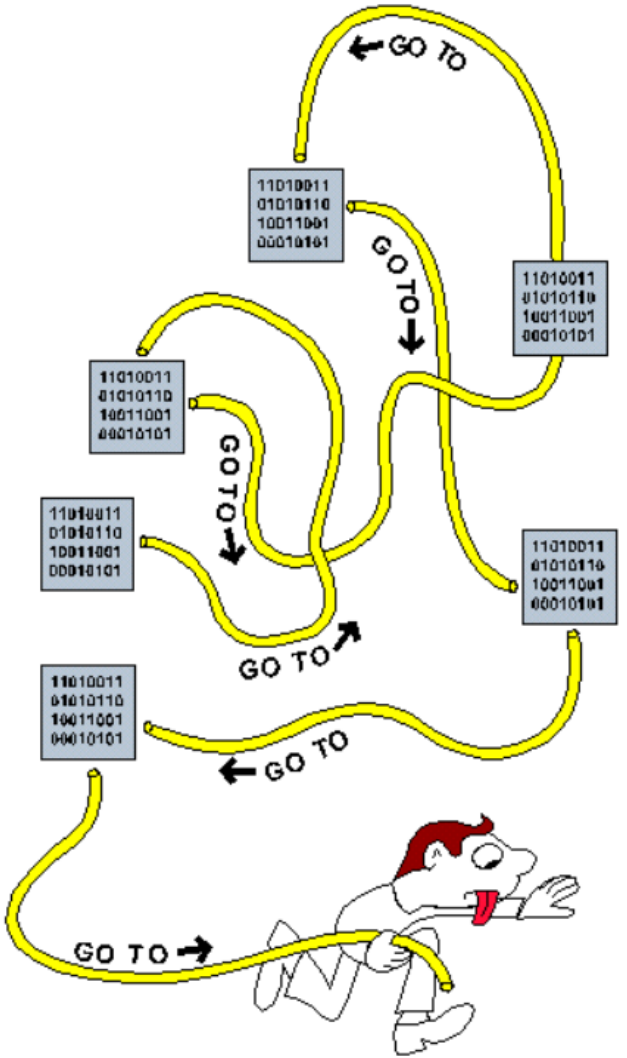
In procedural programming:

- **Data and functions are usually separated**
- Any **function** may access and **modify** different **data**
- **Changes in one part** of the program may affect the **entire system**

In large projects, these issues became more serious:

- Programs often turned into "**Spaghetti Code**"
- Team development became difficult
- **Code reuse** was very limited

Therefore, a new approach was needed that could **combine data and behavior together**.



1970s – Formation of Modern OOP Philosophy

One of the first languages to introduce OOP concepts was:

Simula (1967)

Important features of Simula:

- Introduction of the Class concept
- Creation of Objects from classes
- Ability to model real-world systems
- This language was developed in Norway by:

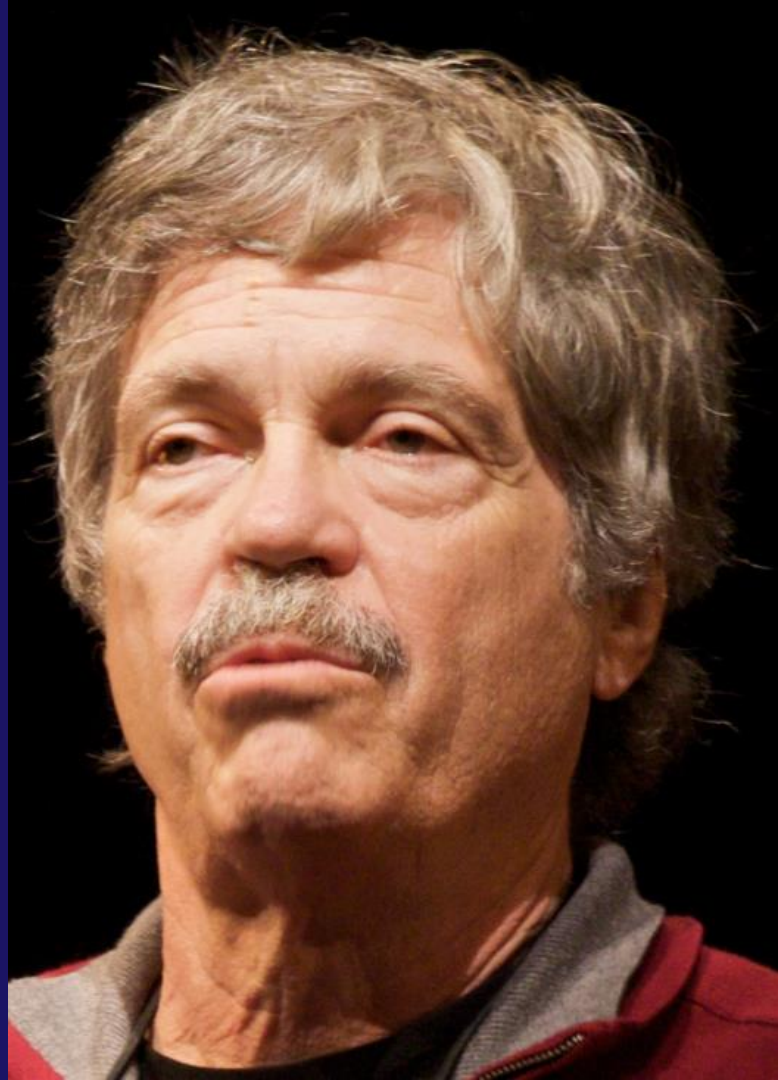
Its main purpose was simulation of complex systems.

1967 – Birth of Early OOP Concepts

One of the most influential figures in the development of OOP is:
Alan Kay
In the **1970s**, he worked at the research center **Xerox PARC**.
His major contributions include:

- Coining the term **Object-Oriented Programming**
- Designing the **Smalltalk** programming language
- Developing the concept of **Message Passing between objects**

His vision was that:
"A program should be composed of **independent objects that communicate by sending messages to each other**."
Smalltalk became one of the most influential languages in the history of OOP.



1. Everything Is An Object.
2. Objects communicate by sending and receiving messages (in terms of objects).
3. Objects have their own memory (in terms of objects).
4. Every object is an instance of a class (which must be an object).
5. The class holds the shared behavior for its instances (in the form of objects in a program list)

Alan Kays

1983 – OOP Enters the Industrial World
C++ - Bjarne Stroustrup

Core Principles of Object-Oriented Programming
As OOP evolved, several key concepts were established:

- **Encapsulation**
Combining data and related functions into a single unit called a class.

- **Inheritance**
Allowing new classes to be created based on existing classes.

- **Polymorphism**
The ability to use a single interface for different behaviors.

- **Abstraction**
Representing complex systems using simplified models.

These concepts made it much easier to manage **large and complex software projects**.

1995 – Global Popularity Explosion
Java

- Fully object-oriented
- Portable (Write Once, Run Anywhere)
- Brought OOP into mainstream software industry

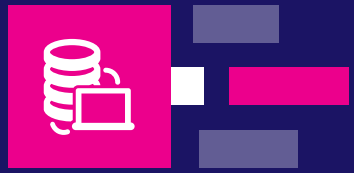
2000s to Present – Evolution and Hybrid Paradigms

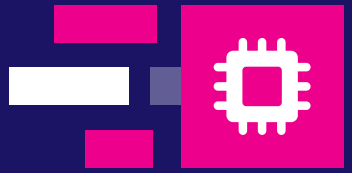
- Rise of C# and .NET
- Growth of Python
- Fusion of OOP with functional programming
- Emphasis on design patterns and software architecture

FINALLY : WHY DO WE NEED OOP?

OOP can make software development more modular

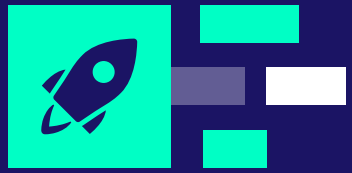
OOP systems can be easily scaled from small to large applications, provided that their design quality supports such growth





Which can make it easier to upgrade and update

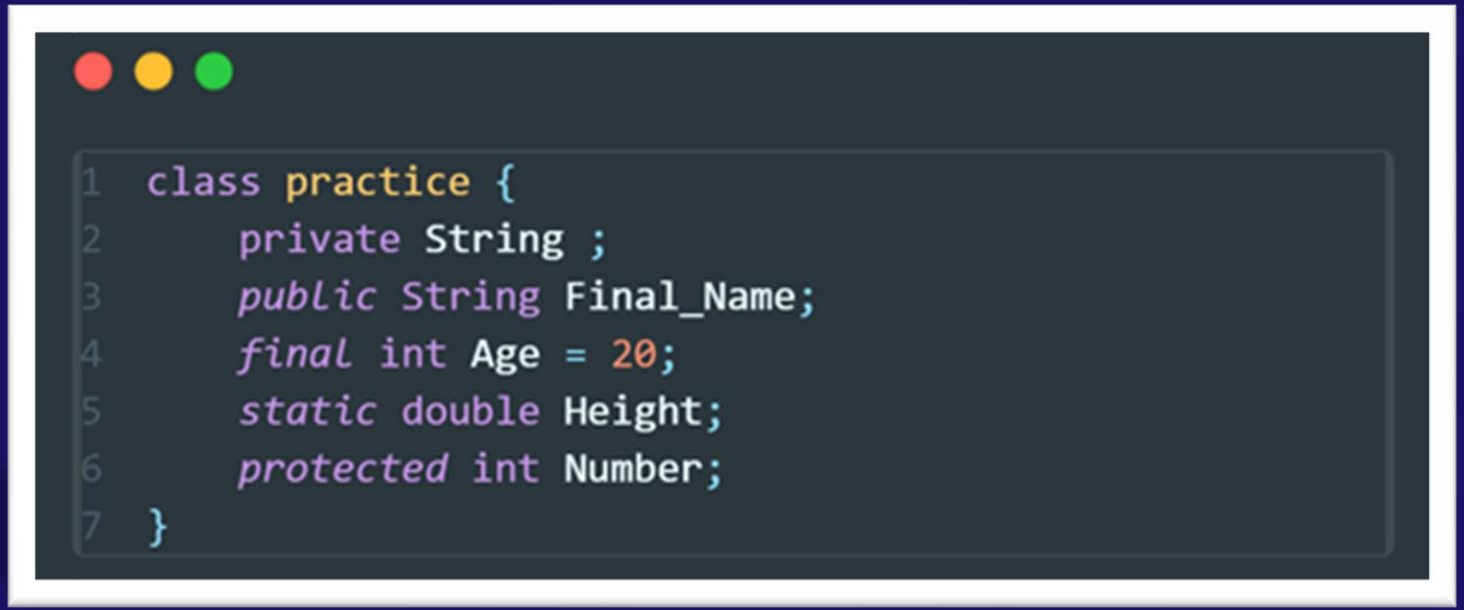
It is very easy to partition the work in a project



Class Attribute

- **Java class attributes** are the variables that are bound in a class
- A class attribute defines the state of the class during program execution
- A class attribute is accessible within class methods by default.

Example

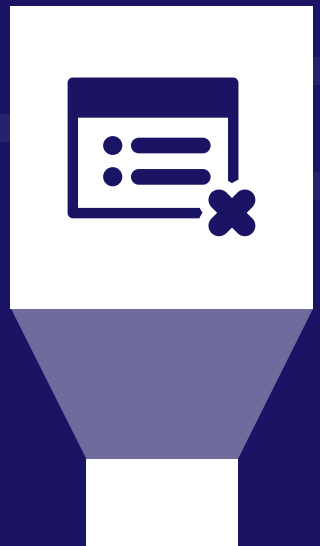


```
class practice {
    private String ;
    public String Final_Name;
    final int Age = 20;
    static double Height;
    protected int Number;
}
```

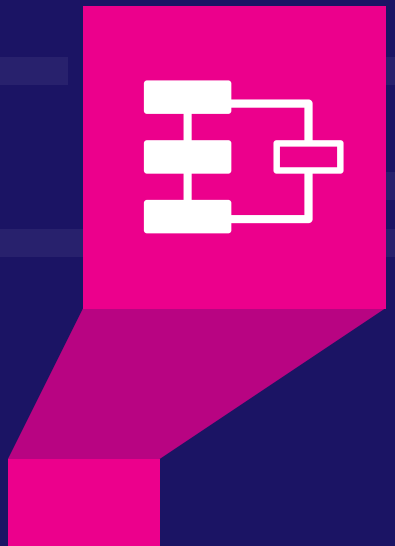
Access Modifires Review

Public

Private



Protected



02

Methods

Class Methods

- A method in Java is a set of instructions that can be called for execution using the method name
- Getters and setters are used to protect your data

Example

```
1       private String ;
2       public String Final_Name;
3       final int Age = 20;
4       static double Height;
5       protected int Number;
6       double Change_Grade(double Grade){
7           return Grade*2;
8       }
9       String add(String Name, String Final_Name){
10          return Name + " " + Final_Name;
11      }
```

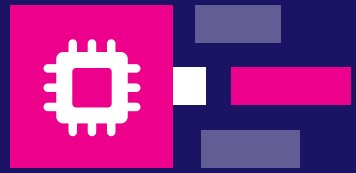
03

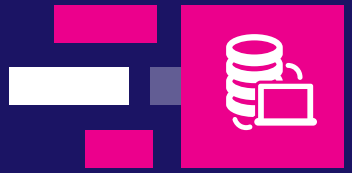
Constructors

Constructors

A constructor is a special method in a class that initializes new objects.

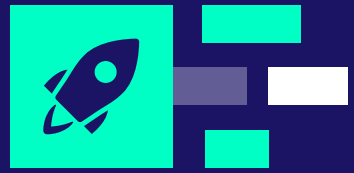
OOP systems can be easily upgraded from small to large systems





In most programming languages, the constructor has the same name as the class.(such as JAVA)

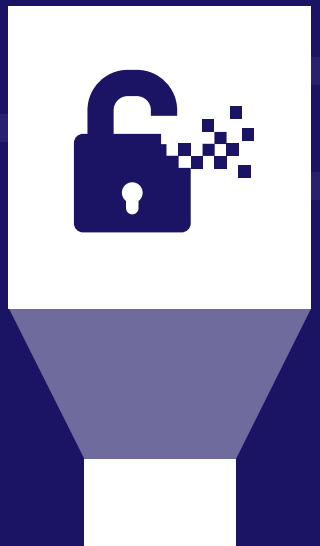
It is very easy to partition the work in a project



What are types of constructors

No Arguments

Default

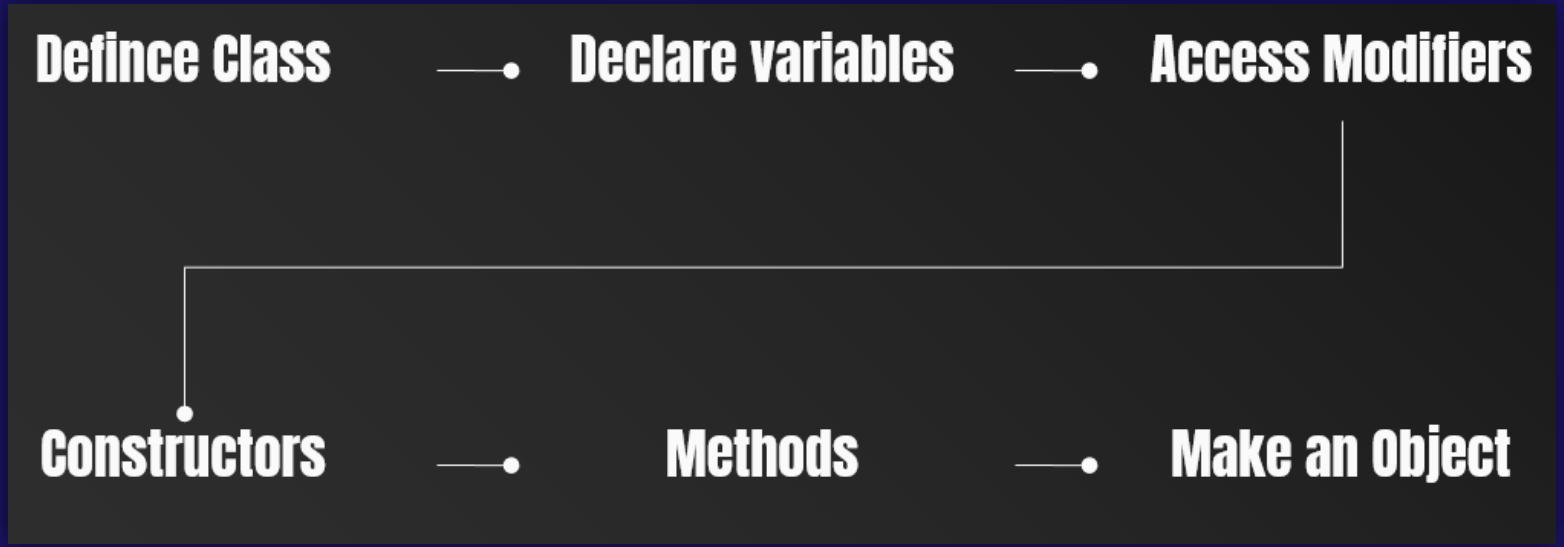


Parametrized

Example

```
1       practice (String n){
2           Name = n;
3       }
4
5       practice(String Name , String Final_Name , double Grade){
6           this.Grade = Grade;
7           this.Final_Name=Final_Name;
8           this.Name = Name;
9       }
10
11      practice(String Name , String Final_Name , int Age){
12          this(Name);
13      }
```

Sequence


Defince Class
Declare variables
Access Modifiers
Constructors
Methods
Make an Object

04

UML Diagram

UML

1

The UML Class diagram is a graphical notation (Unified Modeling Language)

2

Is used to construct and visualize object oriented systems

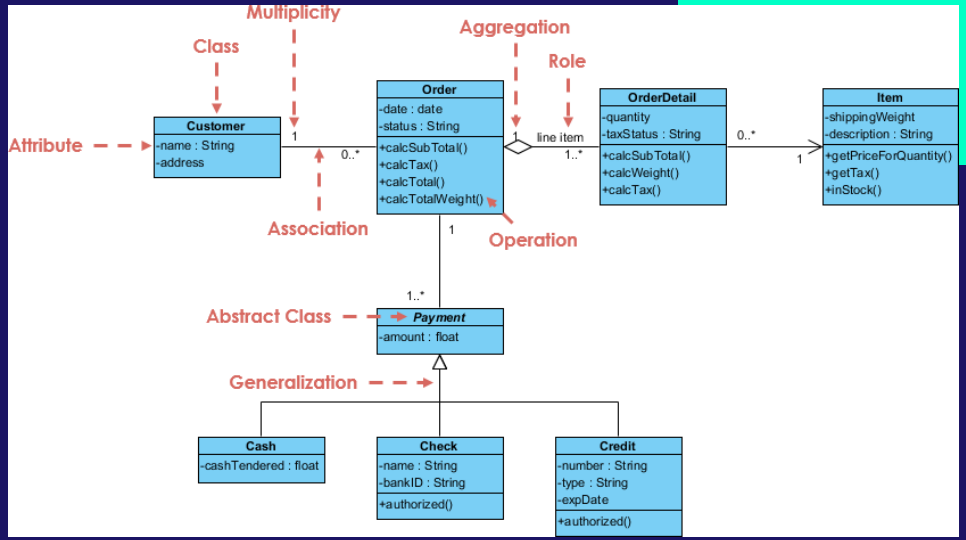
3

Is a type of static structure diagram

4

Describes the structure of a system

Why do we need UML?

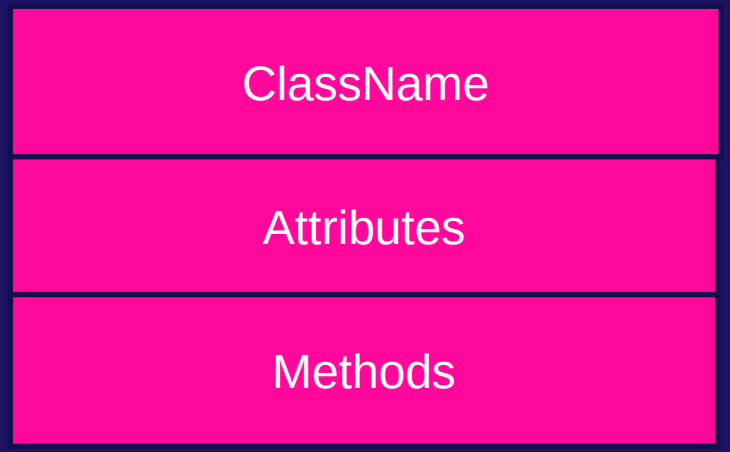


UML provides a standardized way to visualize the design of a system

How to show classes with UML

Info Shown

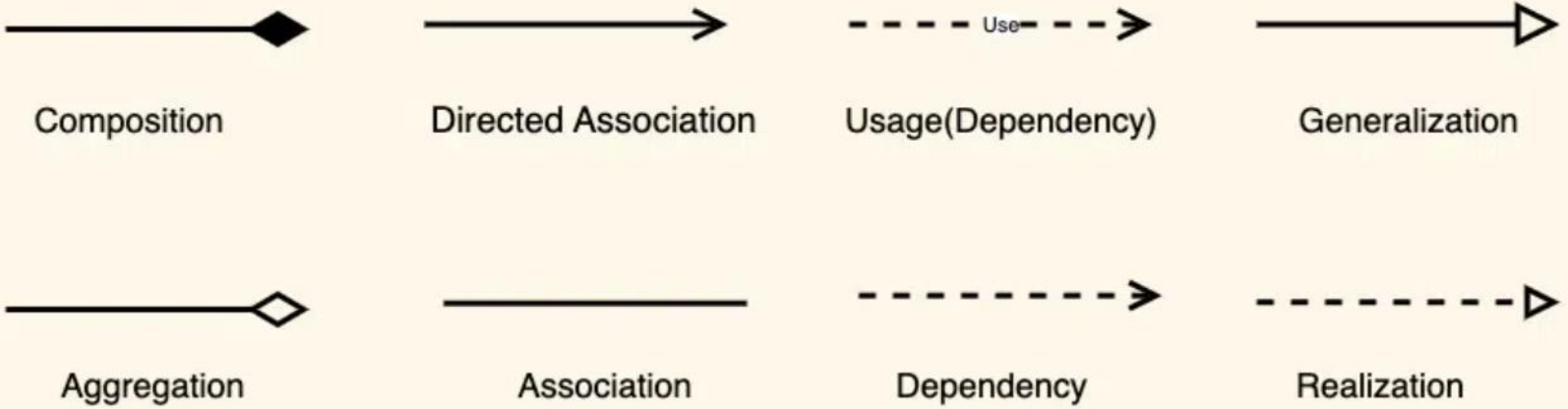
- Variable Names
- Access modifiers
- Methods and Their Arguments



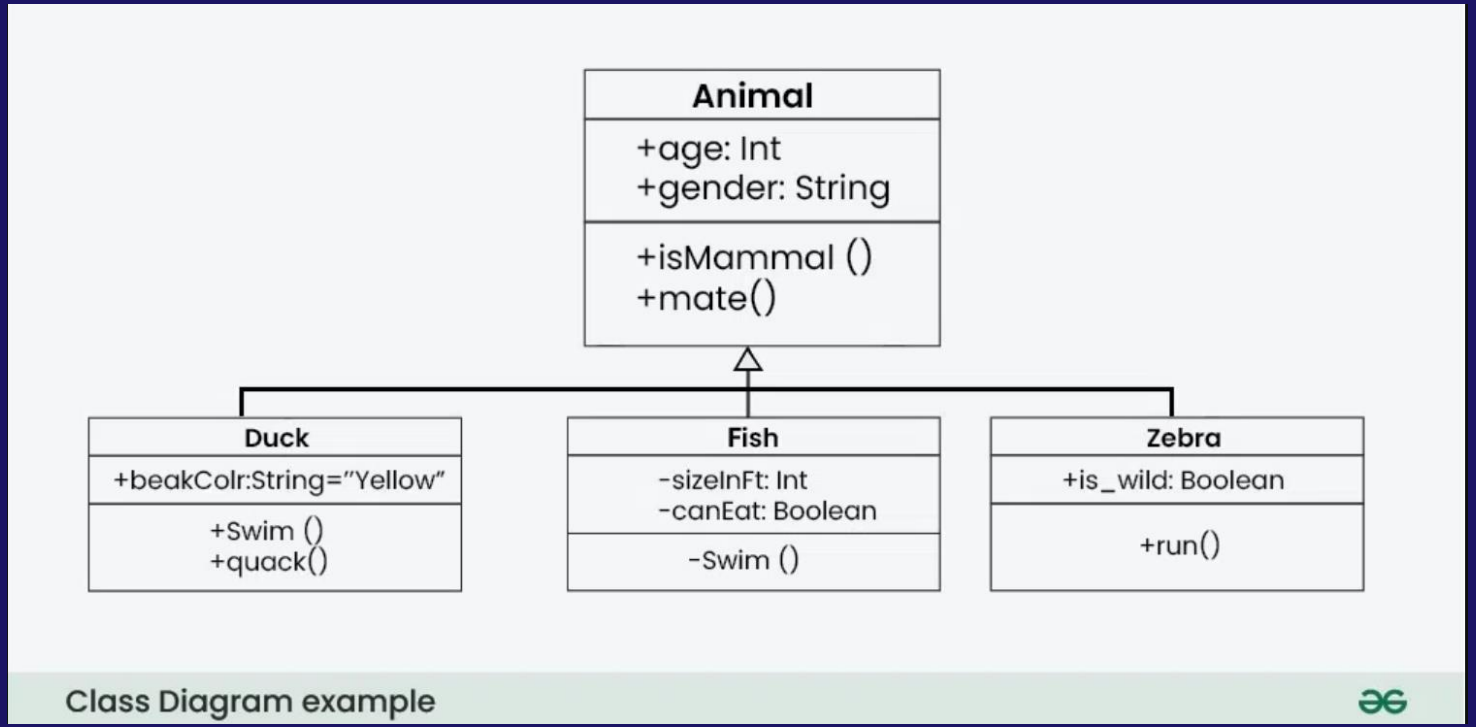
Important Marks

Public	Private	Protected
+	-	#
Package	Derived	Static
~	/	_

Class Diagram Relationships



An example


Animal
+age: Int
+gender: String
+isMammal ()
+mate()
Duck
+beakColr:String="Yellow"
+Swim ()
+quack()
Fish
-sizeInFt: Int
-canEat: Boolean
-Swim ()
Zebra
+is_wild: Boolean
+run()
Class Diagram example

THANKS !

Do you have any questions?
parsa.attaran84@gmail.com
@In_depthSci

CREDITS: This presentation template was created by Slidesgo, incluiding icons by Flaticon, and infographics & images by Freepik.